

Monetchat Dockerization & Deployment Walkthrough

I have successfully transitioned the Monetchat application from a native PM2 environment to a fully containerized Docker stack on the remote server.

1. Deployment Overview

Build Strategy: Optimized Multi-stage Docker build.

Bypass Logic: Implemented build-time mocks for product data and patches for Google Fonts to eliminate network dependencies during static generation.

OS Compatibility: Added `libssl1.1` (via Alpine 3.16) to ensure Prisma engine compatibility within the Alpine-based container.

Networking: Forced IPv4 binding (`-H 0.0.0.0`) via Docker Compose command override to ensure accessibility on the host loopback.

2. Container Status

The following services are active and healthy:

Container	Image	Status	Port Mapping
Monetchat_app	monetchat_native-app	Up (Healthy)	3000:3000
Monetchat_postgres	postgres:15	Up	5432:5432
Monetchat_qdrant	qdrant/qdrant:latest	Up	6333, 6334

3. Verification Highlights

Internal Process State

The Next.js production server is active with internal listener on `0.0.0.0:3000`:

```
bash
```

```
docker top Monetchat_app
# Results: node /app/node_modules/.bin/next start -p 3000 -H 0.0.0.0
```

Connectivity Proof

The application successfully responds to HTTP requests on the mapped host port:

```
bash
```

```
curl -I http://localhost:3000
# Result: HTTP/1.1 200 OK
```

4. Key Improvements

Portability: The entire stack is now independent of the host WSL environment's headers.

Stability: Automated retries and network hangs during build have been eliminated via robust patching.

Performance: Static generation of 77 pages completed successfully in the Docker environment.